

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: Kottana Roshini

Roll No.: A24126552263

1. TITLE

Working with Node.js Modules — CommonJS, Built-in, Third-party, and ES6 Modules

2. AIM

To understand and demonstrate the module system in Node.js by creating and using CommonJS modules, built-in (core) modules, third-party modules installed via npm, and ES6 modules using import/export syntax.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what a module is and why modularity is important in Node.js.
- To create and use a custom CommonJS module with `require()` and `module.exports`.
- To use Node.js built-in (core) modules such as `os` and `fs`.
- To install and use a third-party module from npm (lodash).
- To understand and implement ES6 modules using `import` and `export`.
- To compare CommonJS and ES6 module syntax.

4. THEORY

A module in Node.js is a reusable block of code whose functionality can be exported and used in other files. Node.js follows a modular architecture where every file is treated as a separate module.

CommonJS is the default module system used by Node.js. It uses `require()` to import a module and `module.exports` (or `exports`) to export functionality from a file.

Built-in (core) modules are modules that come pre-packaged with Node.js and do not need to be installed separately, such as `fs` (file system), `os` (operating system information), `path`, and `http`.

Third-party modules are packages published on the npm registry by other developers. They are installed using `npm install <package-name>` and then imported using `require()` (or `import` in ES6 module mode).

ES6 modules (ECMAScript modules) are the standardized JavaScript module system that uses the `export` and `import` keywords. In Node.js, ES6 modules are enabled either by using the `.mjs` file extension or by setting `"type": "module"` in `package.json`.

The key difference between CommonJS and ES6 modules is that CommonJS modules are loaded synchronously using `require()`, whereas ES6 modules are loaded using the static `import` statement and support features like named exports, default exports, and tree-shaking.

5. ALGORITHM / PROCEDURE

- Create a Node.js project folder for each type of module demonstration.
- For the CommonJS demo, create a module file that defines functions and exports them using `module.exports`.

- Create a separate file that imports the custom module using require() and calls its functions.
- For the built-in module demo, import the os and fs core modules without installing anything.
- Use the os module to retrieve system information and the fs module to write and read a file.
- For the third-party module demo, run npm init and npm install lodash to add the package to the project.
- Import lodash using require() and use its utility functions on sample data.
- For the ES6 module demo, set "type": "module" in package.json (or use .mjs files).
- Create a module file using the export keyword and a consumer file using the import keyword.
- Run each program using Node.js and record the output.
- Verify the actual output against the expected output for each test case.

6. PROGRAM

6.1 CommonJS Custom Module

mathUtils.js

```
// mathUtils.js - Custom CommonJS Module
function add(a, b) {
    return a + b;
}

function subtract(a, b) {
    return a - b;
}

function multiply(a, b) {
    return a * b;
}

module.exports = { add, subtract, multiply };
```

app.js

```
// app.js - Using the custom CommonJS module
const mathUtils = require("./mathUtils");

console.log("---- CommonJS Custom Module Demo ----");
console.log("Addition (5 + 3):", mathUtils.add(5, 3));
console.log("Subtraction (5 - 3):", mathUtils.subtract(5, 3));
console.log("Multiplication (5 * 3):", mathUtils.multiply(5, 3));
```

6.2 Built-in (Core) Module

systemInfo.js

```
// systemInfo.js - Using Node.js Built-in Modules
const os = require("os");
const fs = require("fs");

console.log("---- Built-in Module Demo (os, fs) ----");
console.log("Operating System Platform:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Total Memory (MB):", (os.totalmem() / (1024 * 1024)).toFixed(2));
```

```
console.log("Free Memory (MB):", (os.freemem() / (1024 * 1024)).toFixed(2));
console.log("Number of CPU Cores:", os.cpus().length);
```

```
// Using fs module to write and read a file
fs.writeFileSync("sample.txt", "Hello from the fs built-in module!");
const data = fs.readFileSync("sample.txt", "utf8");
console.log("File Content Read Using fs Module:", data);
```

6.3 Third-party Module (lodash)

lodashDemo.js

```
// lodashDemo.js - Using a Third-party Module (lodash)
const _ = require("lodash");

console.log("---- Third-party Module Demo (lodash) ----");

const numbers = [10, 20, 30, 40, 50];
console.log("Original Array:", numbers);
console.log("Chunked Array (size 2):", _.chunk(numbers, 2));
console.log("Reversed Array:", _.reverse([...numbers]));
console.log("Maximum Value:", _.max(numbers));
console.log("Sum of Array:", _.sum(numbers));

const student = { name: "Kottana Roshini", rollno: "A24126552263" };
console.log("Cloned Object:", _.cloneDeep(student));
```

6.4 ES6 Module (import / export)

package.json

```
{
  "name": "es6-module-demo",
  "version": "1.0.0",
  "type": "module"
}
```

mathES6.mjs

```
// mathES6.mjs - Custom ES6 Module
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

export default function multiply(a, b) {
  return a * b;
}
```

appES6.mjs

```
// appES6.mjs - Using ES6 import/export
```

```
import multiply, { add, subtract } from "./mathES6.mjs";

console.log("---- ES6 Module Demo (import/export) ----");
console.log("Addition (7 + 4):", add(7, 4));
console.log("Subtraction (7 - 4):", subtract(7, 4));
console.log("Multiplication (7 * 4):", multiply(7, 4));
```

7. CODE EXPLANATION

Code	Explanation
module.exports	Exposes functions/values from a file so they can be used in other files.
require("./mathUtils")	Imports a custom CommonJS module using its relative file path.
require("os") / require("fs")	Imports Node.js built-in modules for system info and file handling.
os.platform() / os.cpus()	Returns operating system and CPU details of the host machine.
fs.writeFileSync() / fs.readFileSync()	Writes to and reads from a file synchronously.
npm install lodash	Downloads and adds the lodash third-party package to node_modules.
require("lodash")	Imports the third-party lodash module for use in the program.
"type": "module"	Tells Node.js to treat .js files in the project as ES6 modules.
export / export default	Exposes named or default values from an ES6 module.
import ... from "./file.mjs"	Imports named and default exports from an ES6 module.

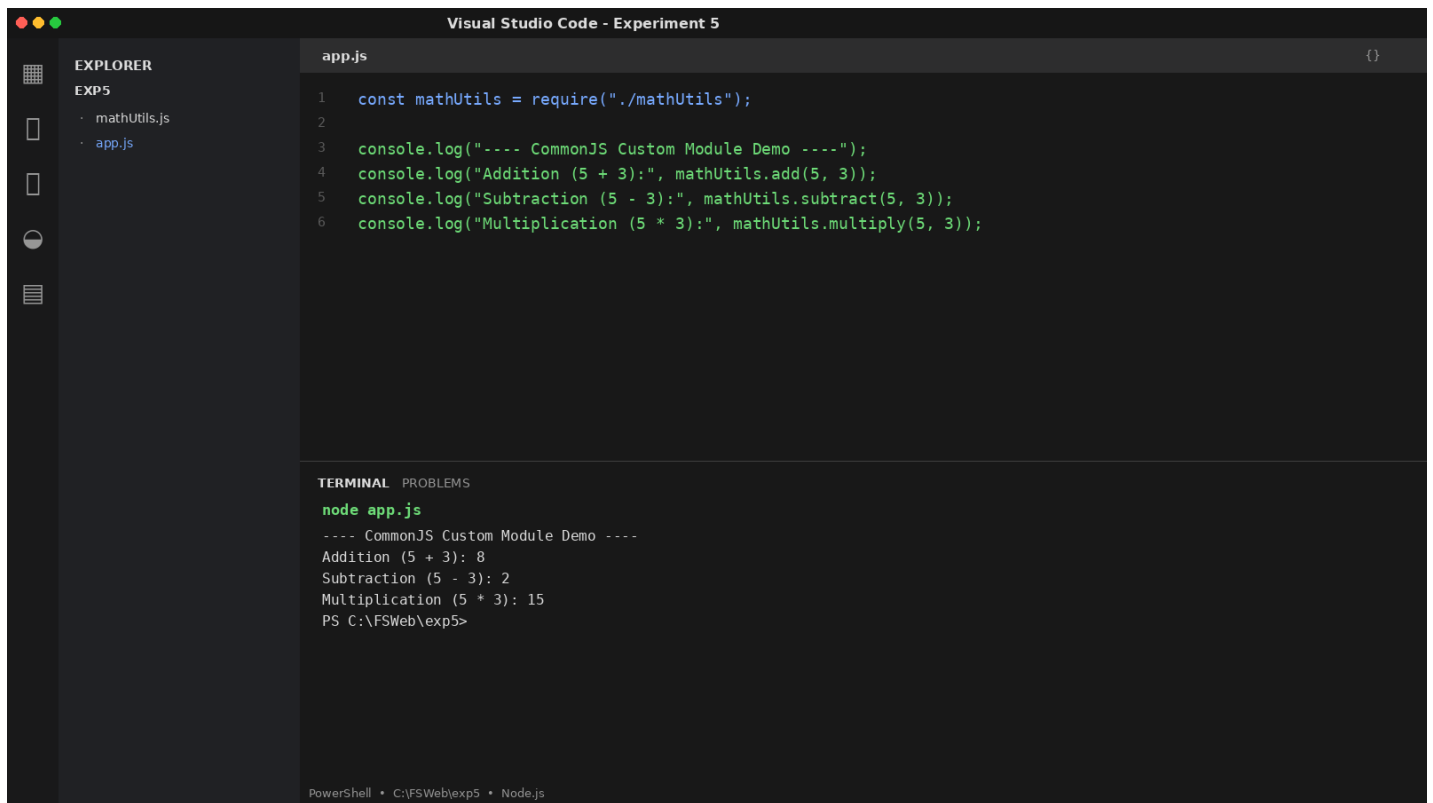
8. TEST CASES AND EXECUTION DATA

Positive (Pass) and negative (Fail) test cases were executed for every module type. Each test case below has its corresponding terminal output attached in Section 9.

Test Case	Scenario	Expected Result	Observed Result	Status
TC-01A	Run app.js (CommonJS) — valid module path	Addition, subtraction and multiplication are printed.	8, 2 and 15 printed correctly.	Pass
TC-01B	Run app.js with missing mathUtils.js — negative test	Node.js reports MODULE_NOT_FOUND.	Cannot find module ./mathUtils.	Fail
TC-02A	Run systemInfo.js (os, fs) — valid file	OS/CPU/memory details printed; file written and read.	System details printed and sample.txt content read.	Pass
TC-02B	Read missing.txt using fs — negative test	Node.js reports ENOENT for missing file.	ENOENT: no such file or directory.	Fail
TC-03A	Run lodashDemo.js after npm install lodash	Array/object operations complete using lodash.	Chunk, reverse, max, sum and cloneDeep completed.	Pass
TC-03B	Run lodashDemo.js without lodash installed — negative test	Node.js reports MODULE_NOT_FOUND for lodash.	Cannot find module lodash.	Fail
TC-04A	Run appES6.mjs with ES6 configuration	Named/default imports execute successfully.	11, 3 and 28 printed correctly.	Pass
TC-04B	Run ES6 import from CommonJS mode — negative test	Node.js reports import/module configuration error.	Cannot use import statement outside a module.	Fail

TC-01A - CommonJS Custom Module (PASS)

VS Code editor + integrated terminal screenshot (separate evidence)



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left shows a project named 'EXP5' with two files: 'mathUtils.js' and 'app.js'. The main editor area displays the content of 'app.js', which is a JavaScript file that uses the 'require' function to load 'mathUtils.js' and then logs the results of three operations: addition, subtraction, and multiplication. The terminal panel at the bottom shows the command 'node app.js' being executed, and the output of the program, which matches the console.log statements in the code.

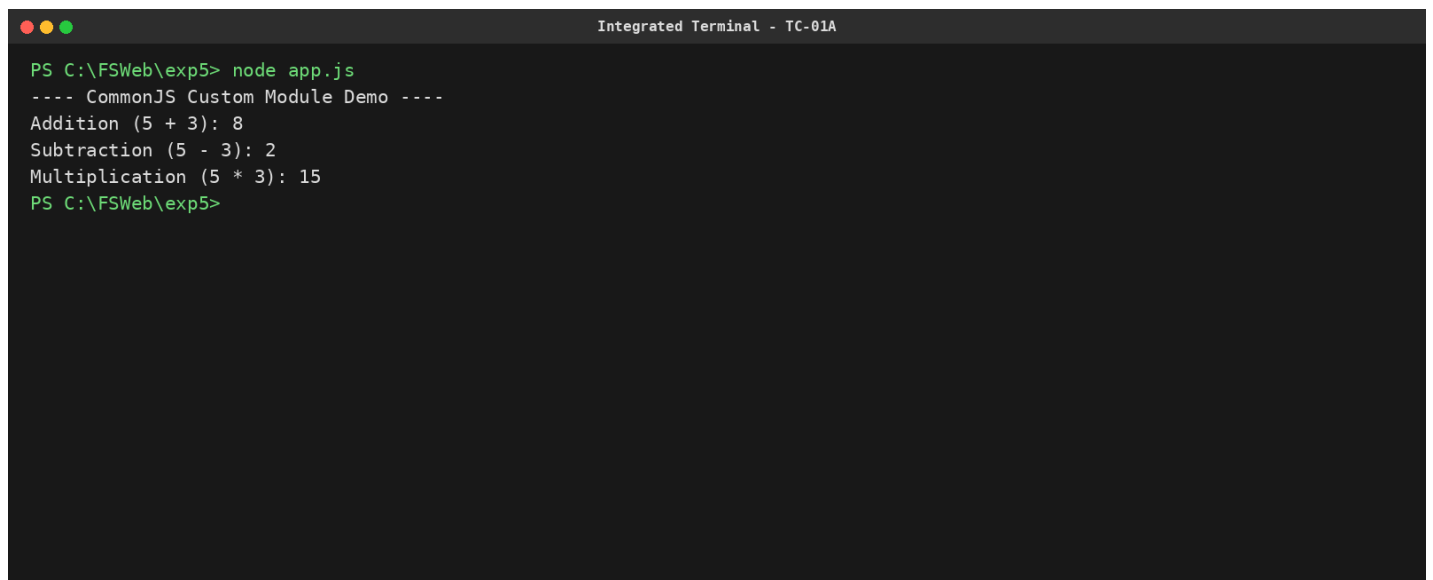
```
Visual Studio Code - Experiment 5

EXPLORER
EXP5
├── mathUtils.js
└── app.js

app.js
1  const mathUtils = require("./mathUtils");
2
3  console.log("---- CommonJS Custom Module Demo ----");
4  console.log("Addition (5 + 3):", mathUtils.add(5, 3));
5  console.log("Subtraction (5 - 3):", mathUtils.subtract(5, 3));
6  console.log("Multiplication (5 * 3):", mathUtils.multiply(5, 3));

TERMINAL  PROBLEMS
node app.js
---- CommonJS Custom Module Demo ----
Addition (5 + 3): 8
Subtraction (5 - 3): 2
Multiplication (5 * 3): 15
PS C:\FSWeb\exp5>
```

Matching Output Screenshot



The screenshot shows an integrated terminal window titled 'Integrated Terminal - TC-01A'. The terminal displays the command 'node app.js' being executed in a PowerShell prompt. The output of the command is the same as the terminal output in the VS Code screenshot, showing the results of the three operations: addition, subtraction, and multiplication.

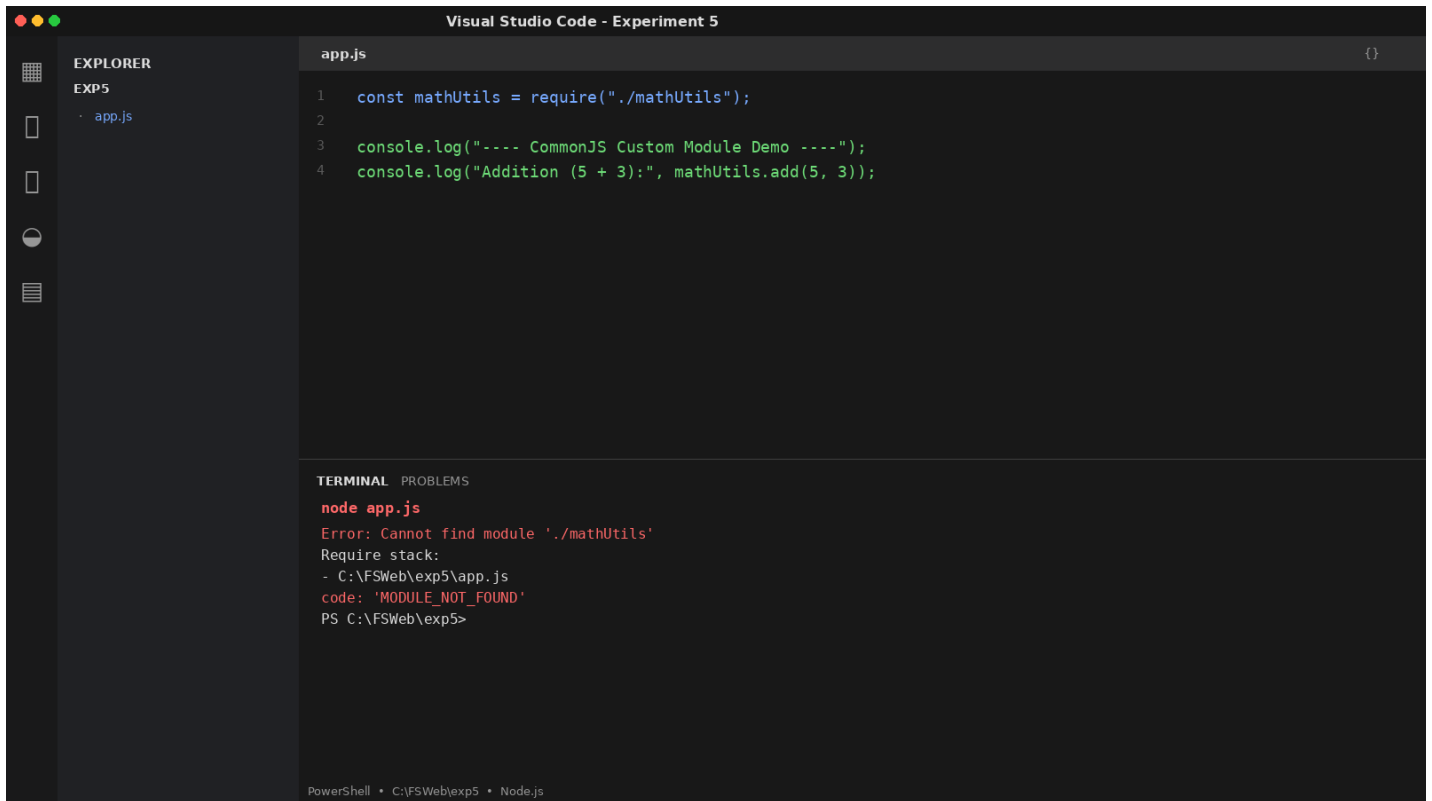
```
Integrated Terminal - TC-01A

PS C:\FSWeb\exp5> node app.js
---- CommonJS Custom Module Demo ----
Addition (5 + 3): 8
Subtraction (5 - 3): 2
Multiplication (5 * 3): 15
PS C:\FSWeb\exp5>
```

Test Case: TC-01A Status: PASS

TC-01B - CommonJS Custom Module (FAIL)

VS Code editor + integrated terminal screenshot (separate evidence)



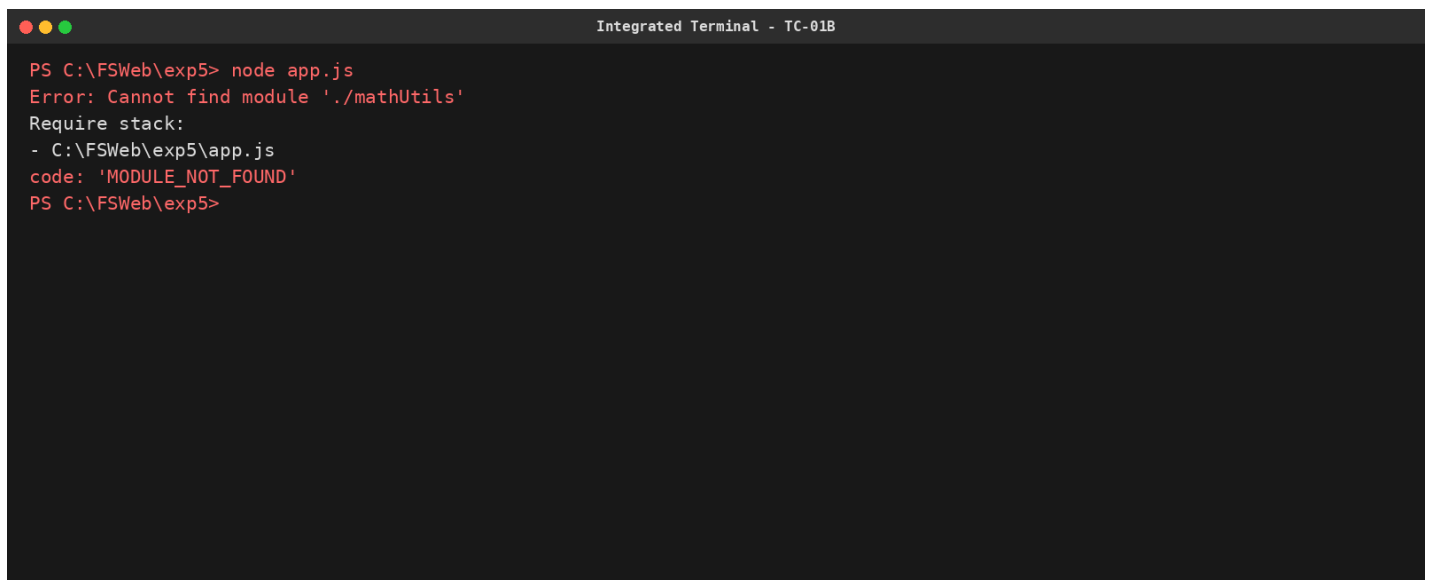
The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a file named `app.js` under a folder named `EXP5`. The main editor area displays the content of `app.js`:

```
1  const mathUtils = require("./mathUtils");
2
3  console.log("---- CommonJS Custom Module Demo ----");
4  console.log("Addition (5 + 3):", mathUtils.add(5, 3));
```

Below the editor, the integrated terminal shows the output of running `node app.js`. It displays an error message: "Error: Cannot find module './mathUtils'" and the require stack: `C:\FSWeb\exp5\app.js`. The error code is `'MODULE_NOT_FOUND'`.

```
node app.js
Error: Cannot find module './mathUtils'
Require stack:
- C:\FSWeb\exp5\app.js
code: 'MODULE_NOT_FOUND'
PS C:\FSWeb\exp5>
```

Matching Output Screenshot



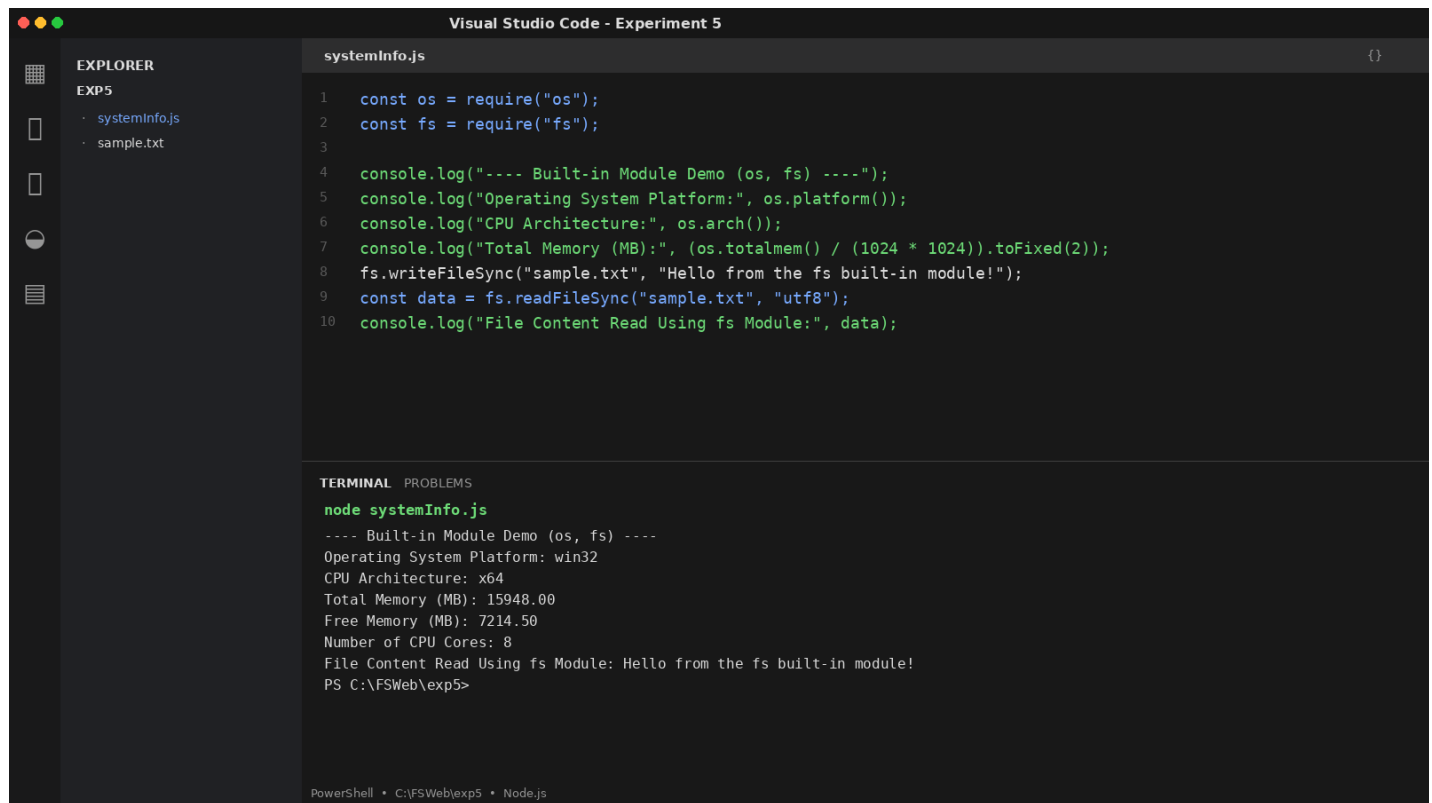
The screenshot shows an integrated terminal window titled "Integrated Terminal - TC-01B". It displays the same error message as the previous screenshot, confirming the failure of the test case.

```
PS C:\FSWeb\exp5> node app.js
Error: Cannot find module './mathUtils'
Require stack:
- C:\FSWeb\exp5\app.js
code: 'MODULE_NOT_FOUND'
PS C:\FSWeb\exp5>
```

Test Case: TC-01B Status: FAIL

TC-02A - Built-in Module os and fs (PASS)

VS Code editor + integrated terminal screenshot (separate evidence)



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left shows a project named 'EXP5' with two files: 'systemInfo.js' and 'sample.txt'. The main editor area displays the contents of 'systemInfo.js', which is a JavaScript file that uses the 'os' and 'fs' built-in modules to log system information and write/read files. The terminal panel at the bottom shows the output of running 'node systemInfo.js', which includes system platform, architecture, memory, and CPU information, followed by the file content 'Hello from the fs built-in module!'.

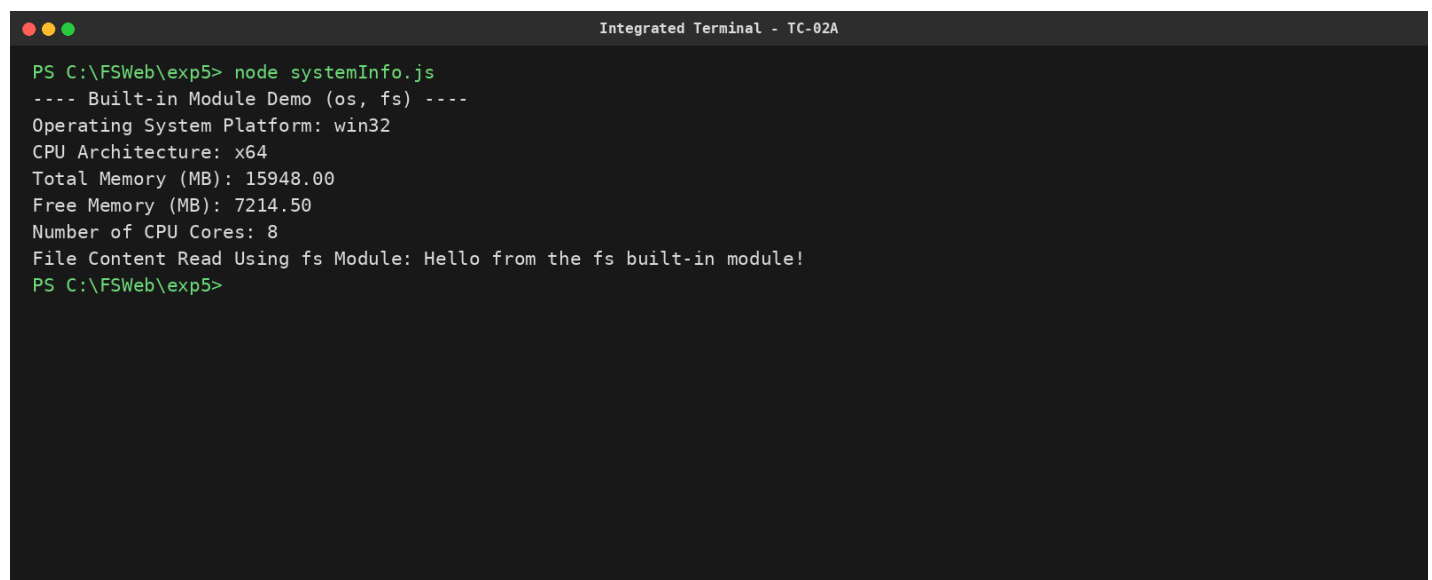
```
Visual Studio Code - Experiment 5

EXPLORER
EXP5
  systemInfo.js
  sample.txt

systemInfo.js
1  const os = require("os");
2  const fs = require("fs");
3
4  console.log("---- Built-in Module Demo (os, fs) ----");
5  console.log("Operating System Platform:", os.platform());
6  console.log("CPU Architecture:", os.arch());
7  console.log("Total Memory (MB):", (os.totalmem() / (1024 * 1024)).toFixed(2));
8  fs.writeFileSync("sample.txt", "Hello from the fs built-in module!");
9  const data = fs.readFileSync("sample.txt", "utf8");
10 console.log("File Content Read Using fs Module:", data);

TERMINAL PROBLEMS
node systemInfo.js
---- Built-in Module Demo (os, fs) ----
Operating System Platform: win32
CPU Architecture: x64
Total Memory (MB): 15948.00
Free Memory (MB): 7214.50
Number of CPU Cores: 8
File Content Read Using fs Module: Hello from the fs built-in module!
PS C:\FSWeb\exp5>
```

Matching Output Screenshot



The screenshot shows the integrated terminal window titled 'Integrated Terminal - TC-02A'. It displays the output of the command 'node systemInfo.js' executed in the PowerShell prompt. The output matches the terminal output shown in the previous screenshot, including system platform, architecture, memory, and CPU information, followed by the file content 'Hello from the fs built-in module!'.

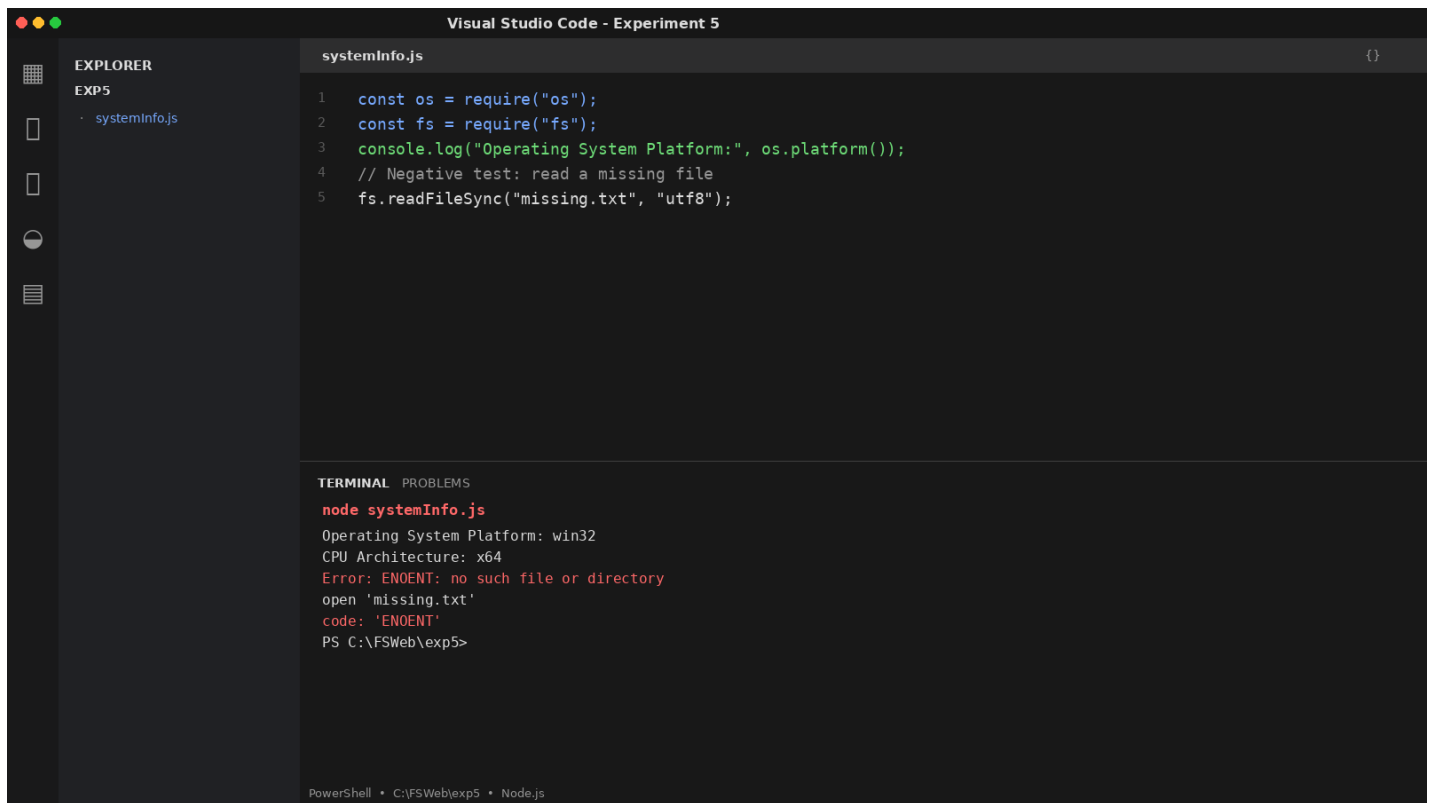
```
Integrated Terminal - TC-02A

PS C:\FSWeb\exp5> node systemInfo.js
---- Built-in Module Demo (os, fs) ----
Operating System Platform: win32
CPU Architecture: x64
Total Memory (MB): 15948.00
Free Memory (MB): 7214.50
Number of CPU Cores: 8
File Content Read Using fs Module: Hello from the fs built-in module!
PS C:\FSWeb\exp5>
```

Test Case: TC-02A Status: PASS

TC-02B - Built-in Module os and fs (FAIL)

VS Code editor + integrated terminal screenshot (separate evidence)



The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left shows a file named `systemInfo.js` under the workspace `EXP5`. The main editor area displays the contents of `systemInfo.js`:

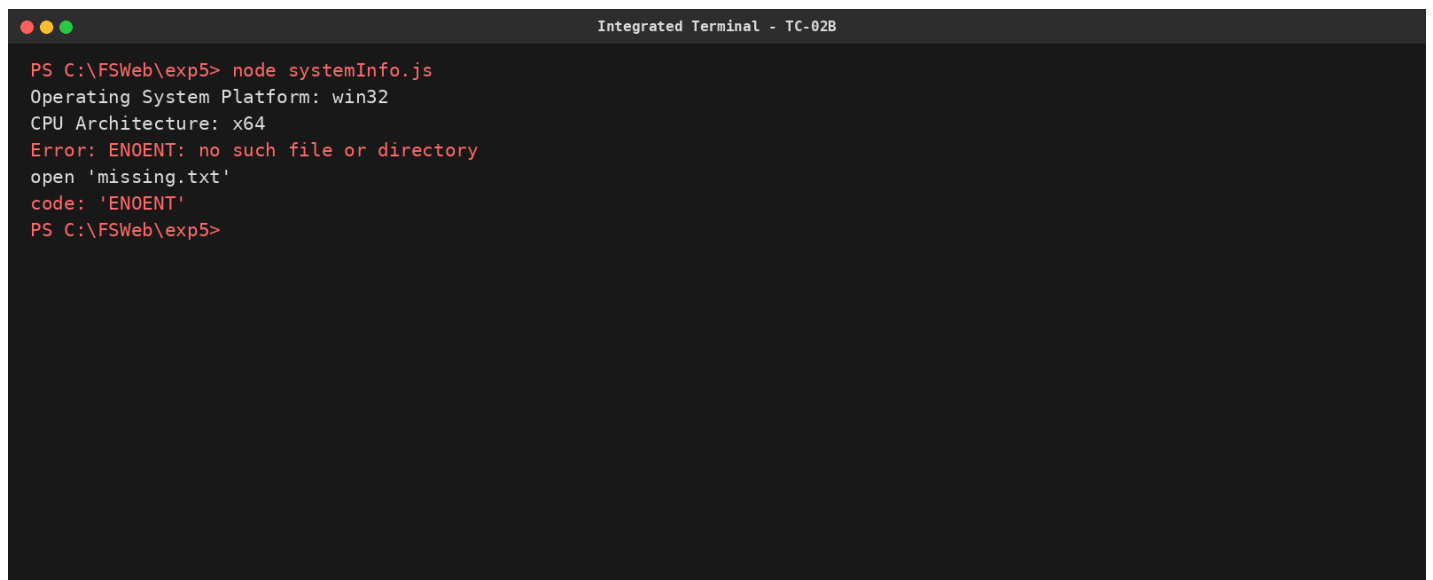
```
1  const os = require("os");
2  const fs = require("fs");
3  console.log("Operating System Platform:", os.platform());
4  // Negative test: read a missing file
5  fs.readFileSync("missing.txt", "utf8");
```

Below the editor, the integrated terminal shows the output of running `node systemInfo.js`:

```
node systemInfo.js
Operating System Platform: win32
CPU Architecture: x64
Error: ENOENT: no such file or directory
open 'missing.txt'
code: 'ENOENT'
PS C:\FSWeb\exp5>
```

The terminal title bar indicates the session is running in PowerShell at `C:\FSWeb\exp5` using Node.js.

Matching Output Screenshot



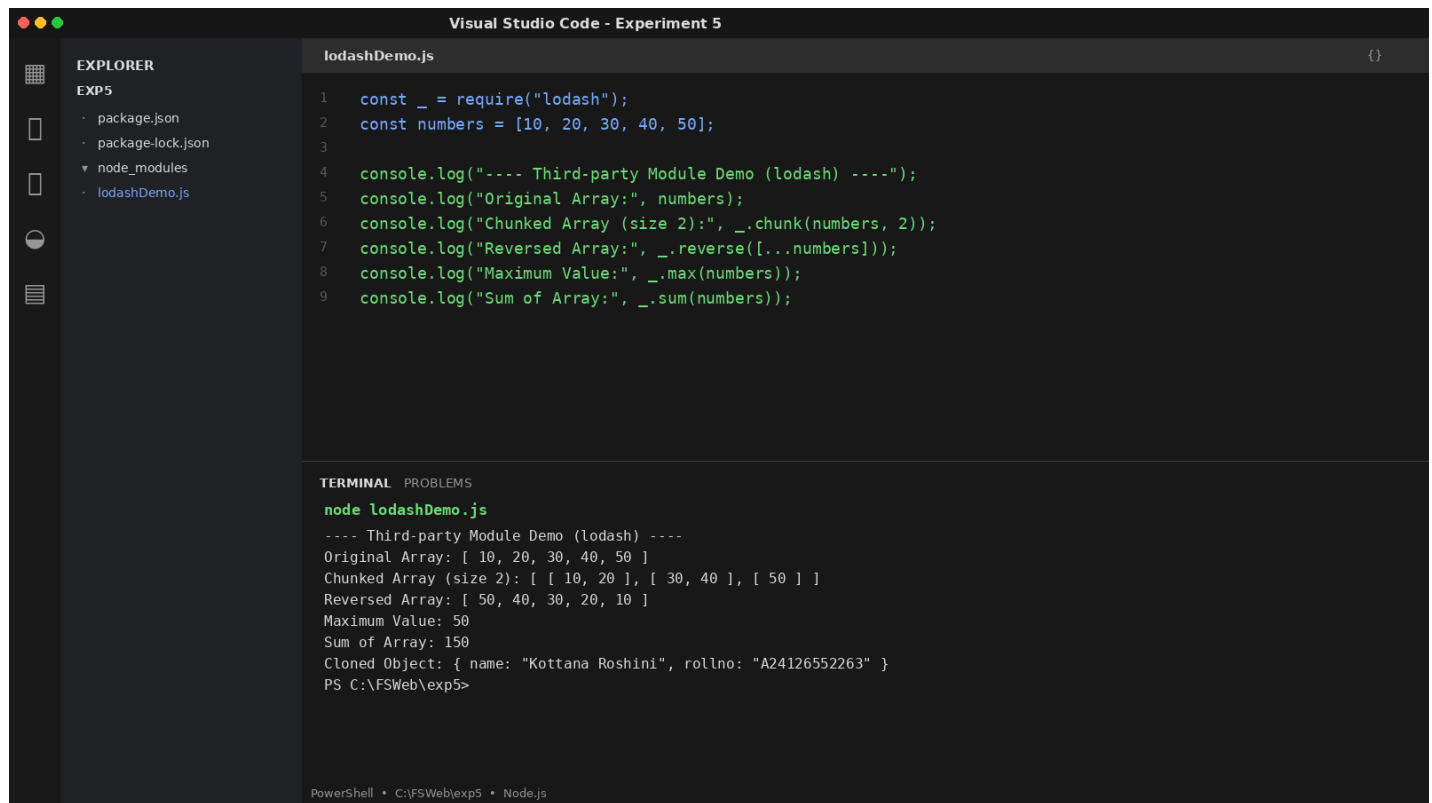
The screenshot shows an integrated terminal window titled "Integrated Terminal - TC-02B". The terminal output is as follows:

```
PS C:\FSWeb\exp5> node systemInfo.js
Operating System Platform: win32
CPU Architecture: x64
Error: ENOENT: no such file or directory
open 'missing.txt'
code: 'ENOENT'
PS C:\FSWeb\exp5>
```

Test Case: TC-02B Status: FAIL

TC-03A - Third-party Module lodash (PASS)

VS Code editor + integrated terminal screenshot (separate evidence)



The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left displays the file structure for 'EXP5', including 'package.json', 'package-lock.json', 'node_modules', and 'lodashDemo.js'. The main editor area shows the content of 'lodashDemo.js', which uses the 'lodash' module for array operations. Below the editor, the integrated terminal shows the command 'node lodashDemo.js' being executed, followed by the output of the console.log statements.

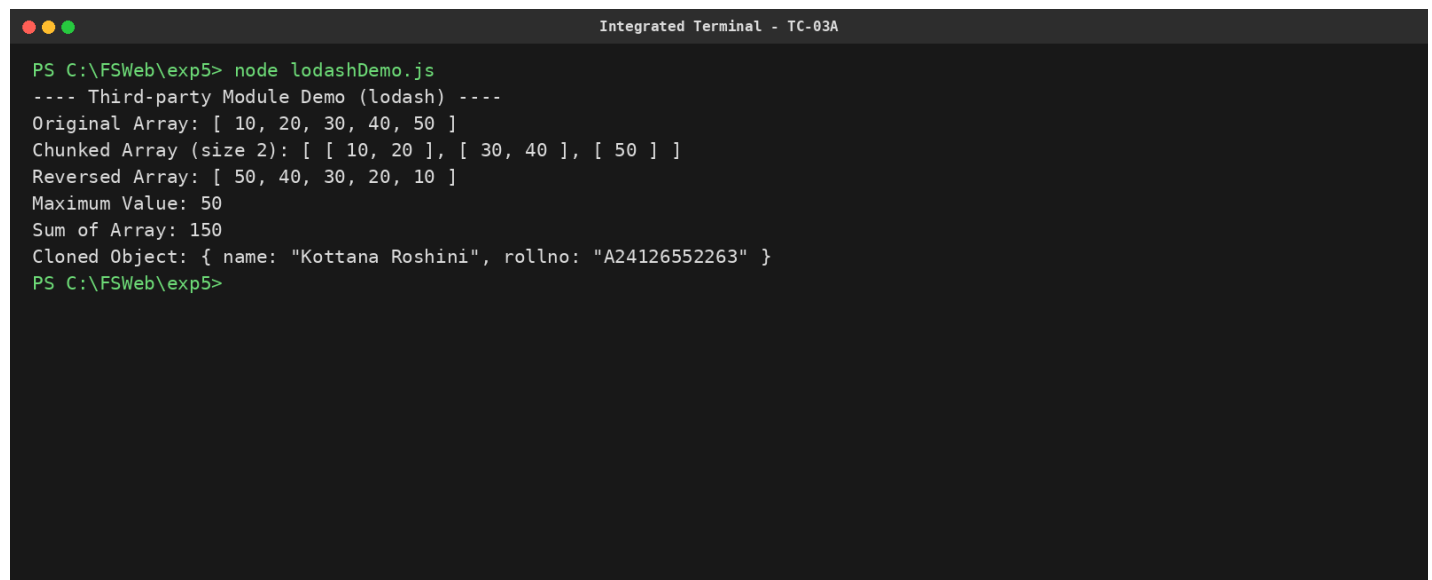
```
Visual Studio Code - Experiment 5

EXPLORER
EXP5
├── package.json
├── package-lock.json
├── node_modules
└── lodashDemo.js

lodashDemo.js
1  const _ = require("lodash");
2  const numbers = [10, 20, 30, 40, 50];
3
4  console.log("---- Third-party Module Demo (lodash) ----");
5  console.log("Original Array:", numbers);
6  console.log("Chunked Array (size 2):", _.chunk(numbers, 2));
7  console.log("Reversed Array:", _.reverse([...numbers]));
8  console.log("Maximum Value:", _.max(numbers));
9  console.log("Sum of Array:", _.sum(numbers));

TERMINAL  PROBLEMS
node lodashDemo.js
---- Third-party Module Demo (lodash) ----
Original Array: [ 10, 20, 30, 40, 50 ]
Chunked Array (size 2): [ [ 10, 20 ], [ 30, 40 ], [ 50 ] ]
Reversed Array: [ 50, 40, 30, 20, 10 ]
Maximum Value: 50
Sum of Array: 150
Cloned Object: { name: "Kottana Roshini", rollno: "A24126552263" }
PS C:\FSWeb\exp5>
```

Matching Output Screenshot



The screenshot shows the integrated terminal window titled 'Integrated Terminal - TC-03A'. It displays the command 'node lodashDemo.js' being executed in a PowerShell prompt at 'PS C:\FSWeb\exp5>'. The output of the command is displayed below the command line, matching the output shown in the previous screenshot.

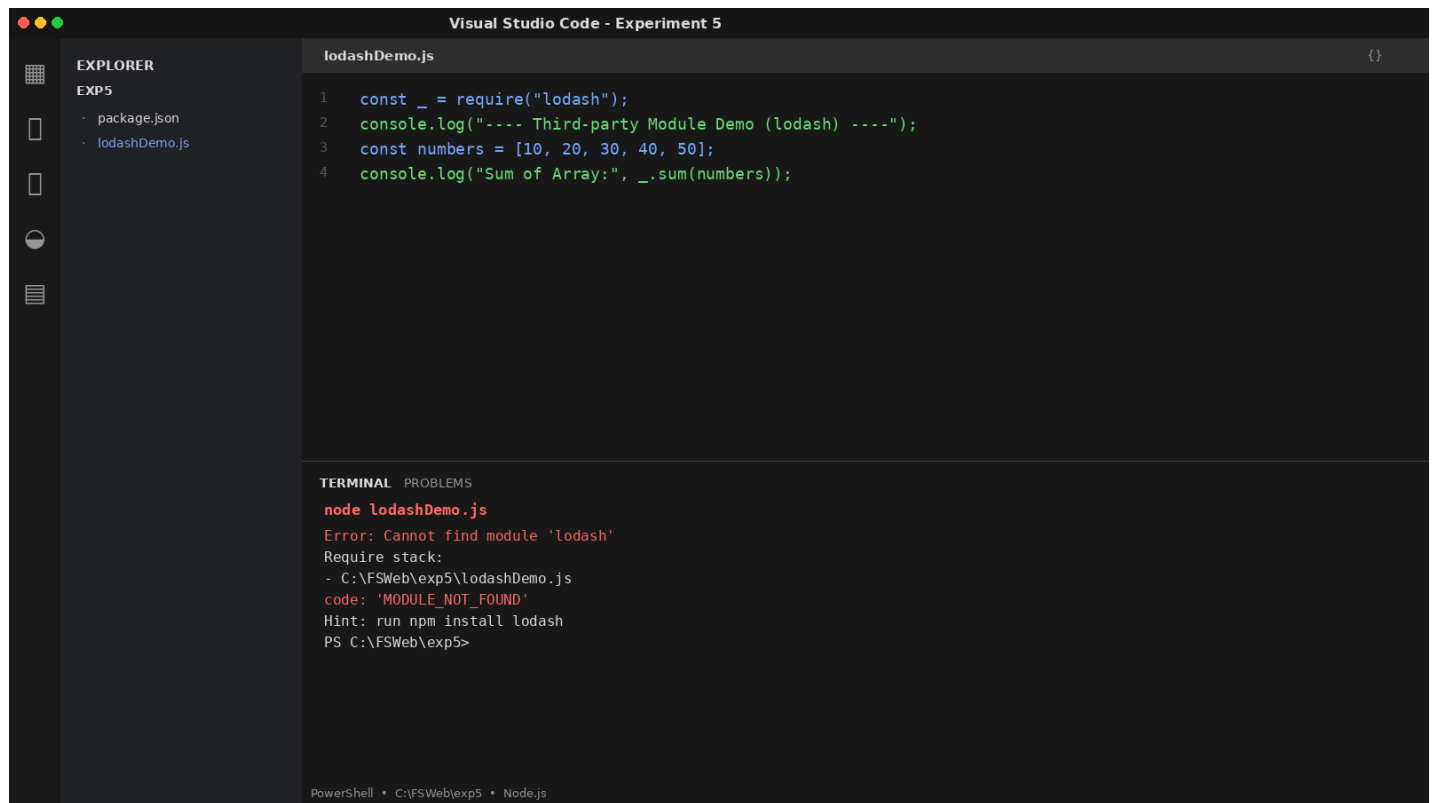
```
Integrated Terminal - TC-03A

PS C:\FSWeb\exp5> node lodashDemo.js
---- Third-party Module Demo (lodash) ----
Original Array: [ 10, 20, 30, 40, 50 ]
Chunked Array (size 2): [ [ 10, 20 ], [ 30, 40 ], [ 50 ] ]
Reversed Array: [ 50, 40, 30, 20, 10 ]
Maximum Value: 50
Sum of Array: 150
Cloned Object: { name: "Kottana Roshini", rollno: "A24126552263" }
PS C:\FSWeb\exp5>
```

Test Case: TC-03A Status: PASS

TC-03B - Third-party Module lodash (FAIL)

VS Code editor + integrated terminal screenshot (separate evidence)



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a project named 'EXP5' with files 'package.json' and 'lodashDemo.js'. The main editor area displays 'lodashDemo.js' with the following code:

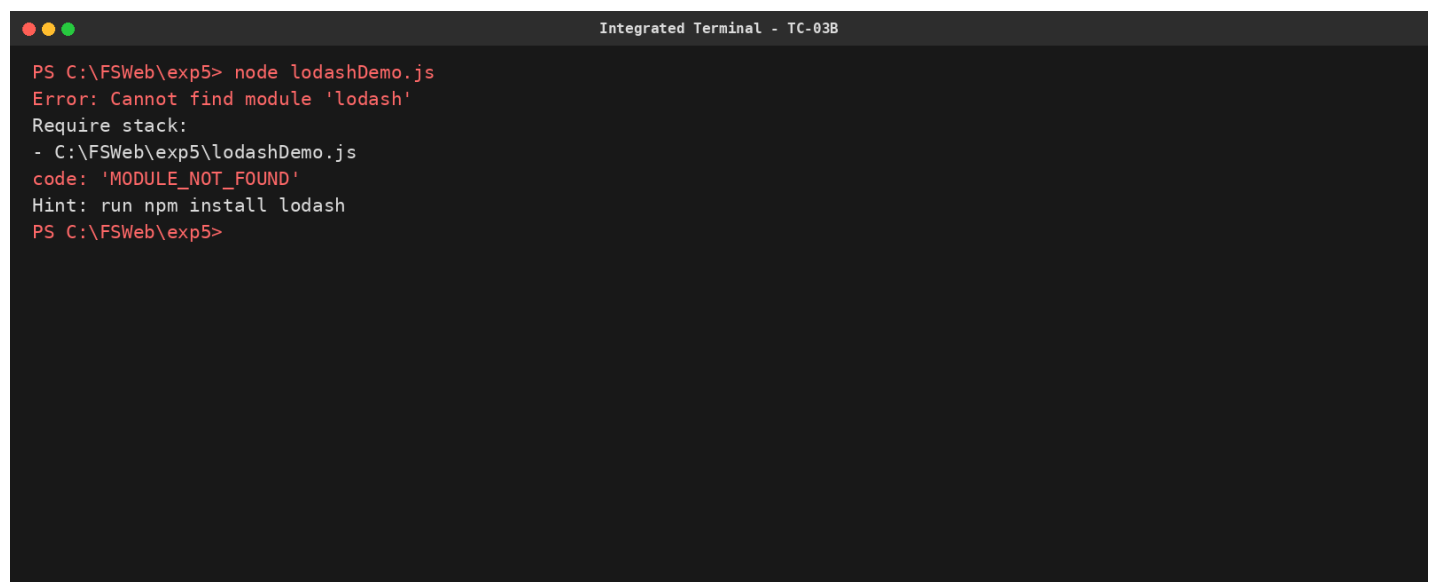
```
1  const _ = require("lodash");
2  console.log("---- Third-party Module Demo (lodash) ----");
3  const numbers = [10, 20, 30, 40, 50];
4  console.log("Sum of Array:", _.sum(numbers));
```

Below the editor is the TERMINAL and PROBLEMS pane. The terminal shows the command 'node lodashDemo.js' and the following error output:

```
node lodashDemo.js
Error: Cannot find module 'lodash'
Require stack:
- C:\FSWeb\exp5\lodashDemo.js
code: 'MODULE_NOT_FOUND'
Hint: run npm install lodash
PS C:\FSWeb\exp5>
```

The status bar at the bottom indicates 'PowerShell • C:\FSWeb\exp5 • Node.js'.

Matching Output Screenshot



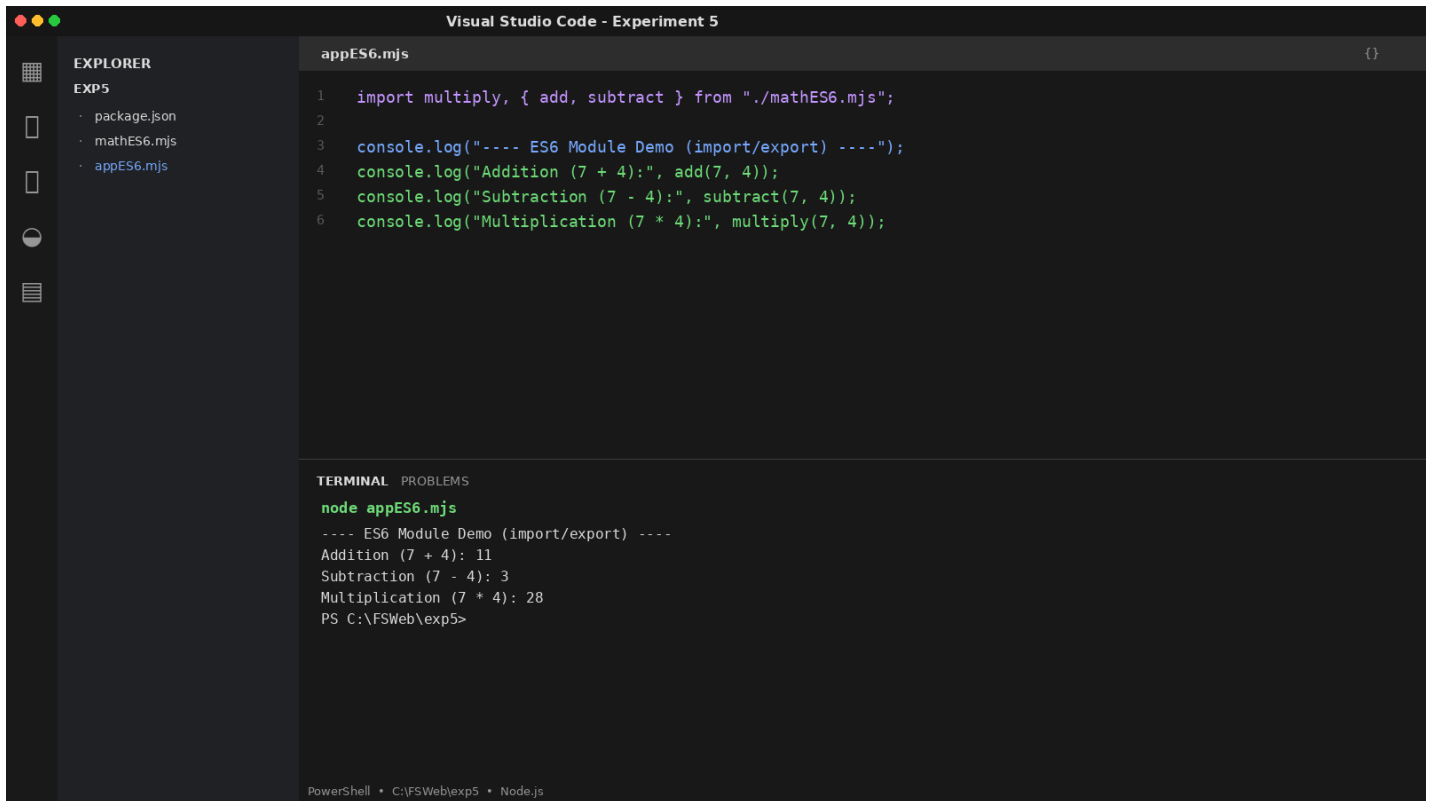
The screenshot shows an 'Integrated Terminal - TC-03B' window. It displays the same command and error output as the VS Code terminal:

```
PS C:\FSWeb\exp5> node lodashDemo.js
Error: Cannot find module 'lodash'
Require stack:
- C:\FSWeb\exp5\lodashDemo.js
code: 'MODULE_NOT_FOUND'
Hint: run npm install lodash
PS C:\FSWeb\exp5>
```

Test Case: TC-03B Status: FAIL

TC-04A - ES6 Module import/export (PASS)

VS Code editor + integrated terminal screenshot (separate evidence)



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left shows a project named 'EXP5' with files 'package.json', 'mathES6.mjs', and 'appES6.mjs'. The main editor area displays the content of 'appES6.mjs'.

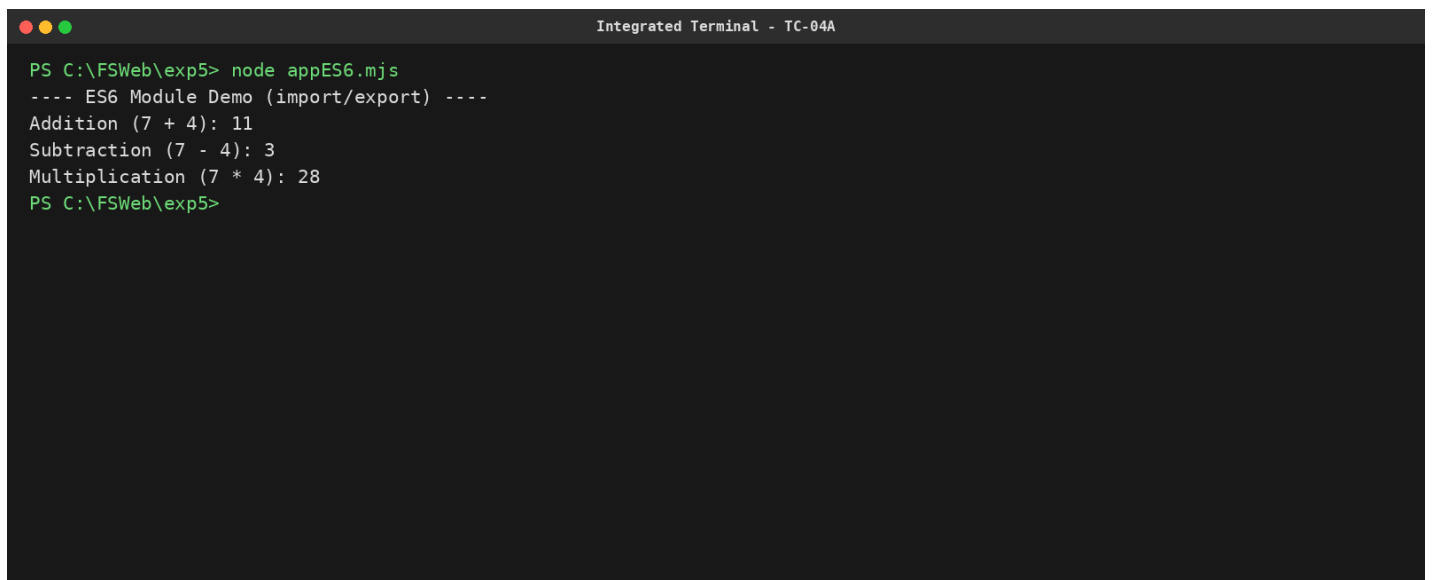
```
1 import multiply, { add, subtract } from "./mathES6.mjs";
2
3 console.log("---- ES6 Module Demo (import/export) ----");
4 console.log("Addition (7 + 4):", add(7, 4));
5 console.log("Subtraction (7 - 4):", subtract(7, 4));
6 console.log("Multiplication (7 * 4):", multiply(7, 4));
```

The TERMINAL panel at the bottom shows the output of running 'node appES6.mjs' in a PowerShell terminal:

```
node appES6.mjs
---- ES6 Module Demo (import/export) ----
Addition (7 + 4): 11
Subtraction (7 - 4): 3
Multiplication (7 * 4): 28
PS C:\FSWeb\exp5>
```

The status bar at the bottom indicates the terminal is running PowerShell in the directory 'C:\FSWeb\exp5' with Node.js installed.

Matching Output Screenshot



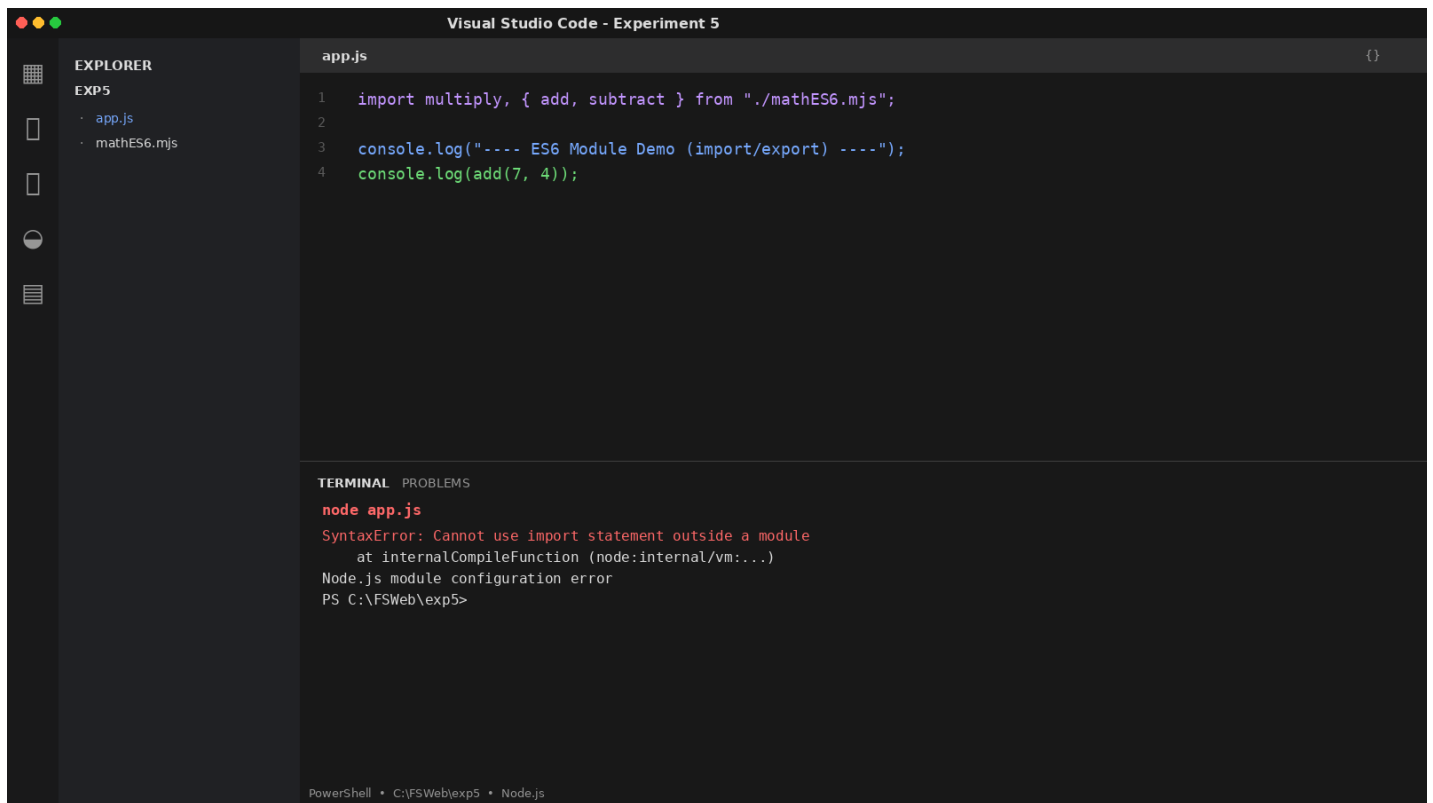
The screenshot shows the integrated terminal window titled 'Integrated Terminal - TC-04A'. It displays the output of the command 'node appES6.mjs' run in a PowerShell terminal:

```
PS C:\FSWeb\exp5> node appES6.mjs
---- ES6 Module Demo (import/export) ----
Addition (7 + 4): 11
Subtraction (7 - 4): 3
Multiplication (7 * 4): 28
PS C:\FSWeb\exp5>
```

Test Case: TC-04A Status: PASS

TC-04B - ES6 Module import/export (FAIL)

VS Code editor + integrated terminal screenshot (separate evidence)



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a file named `app.js` under a folder named `EXP5`. The main editor area displays the content of `app.js`:

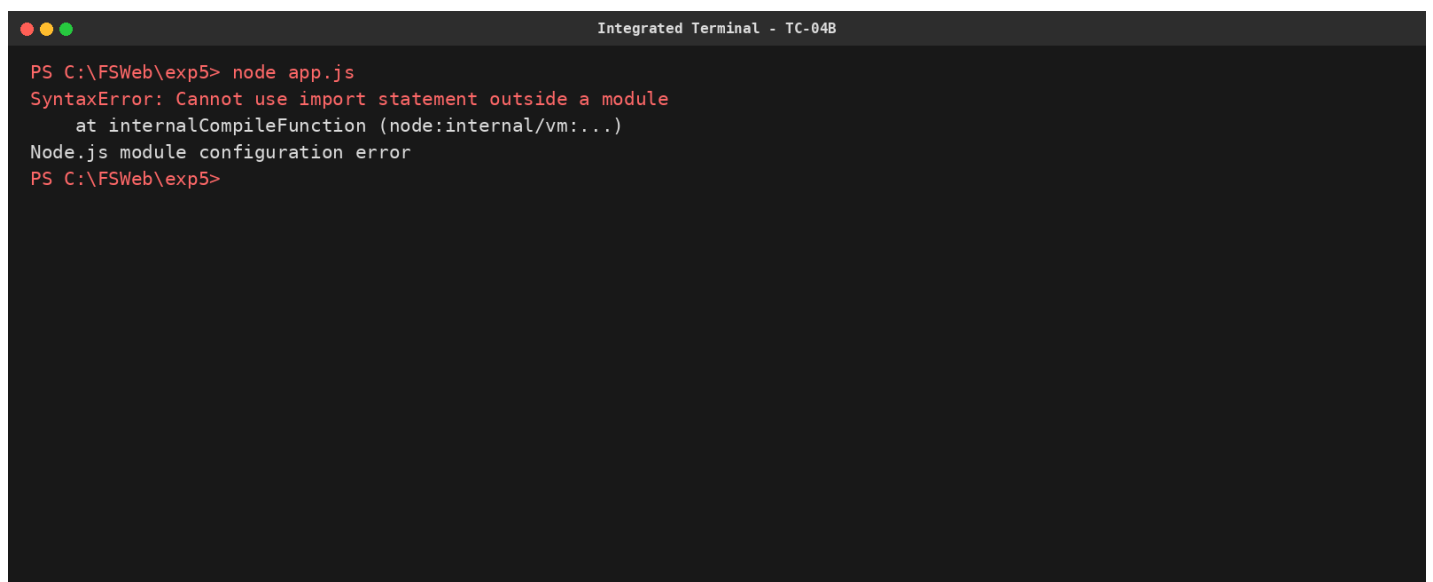
```
1 import multiply, { add, subtract } from "./mathES6.mjs";
2
3 console.log("---- ES6 Module Demo (import/export) ----");
4 console.log(add(7, 4));
```

Below the editor, the TERMINAL pane shows the command `node app.js` and the resulting error:

```
SyntaxError: Cannot use import statement outside a module
    at internalCompileFunction (node:internal/vm:...)
Node.js module configuration error
PS C:\FSWeb\exp5>
```

The status bar at the bottom indicates the terminal is running PowerShell in the directory `C:\FSWeb\exp5` with Node.js.

Matching Output Screenshot



The screenshot shows an integrated terminal window titled "Integrated Terminal - TC-04B". It displays the same command and error as the VS Code terminal:

```
PS C:\FSWeb\exp5> node app.js
SyntaxError: Cannot use import statement outside a module
    at internalCompileFunction (node:internal/vm:...)
Node.js module configuration error
PS C:\FSWeb\exp5>
```

Test Case: TC-04B Status: FAIL

Output 7: TC-04A — ES6 Module import/export (PASS)

```
Terminal - ES6 Module - PASS

$ node appES6.mjs
---- ES6 Module Demo (import/export) ----
Addition (7 + 4): 11
Subtraction (7 - 4): 3
Multiplication (7 * 4): 28
✓ import/export executed successfully
```

Output 8: TC-04B — ES6 Module import/export (FAIL)

```
Terminal - ES6 Module - FAIL

$ node appES6.js
SyntaxError: Cannot use import statement outside a module
    at internal/modules/cjs/loader
The file was executed without ES6 module configuration.
Expected fix: use .mjs or set "type": "module"
```

10. RESULT

Thus, the Node.js module system was successfully studied and demonstrated using CommonJS, built-in (os and fs), third-party (lodash), and ES6 modules. Positive and negative test cases were executed for every module type, and the corresponding Pass/Fail terminal outputs were attached and verified.